

Database Lab Task 1: Assessment

Patsamatla Bhaavya Ramita, 25MCB1010
M.Tech Computer Science and Engineering (Big Data Analytics)

Database Modelling and Design Lab

Scenario: **NutriFit: Health-centric Food Inventory Management**

The project simulates a real-world scenario of Food inventory management system considering users nutritional needs and food preferences. The database consists of three tables:

1. **Users:** The table holds users contact information for expiry reminders, food preferences and nutritional needs.
2. **SupplierDetails:** The table holds the products' market information including name of the grocers, average shopping price, location and category of foods purchased.
3. **FoodInventory:** The table holds the inventory details, including their primary nutrients, food categories and expiry dates. It includes foreign keys from Users and SupplierDetails tables to establish a relationship among them.

Creation of Database

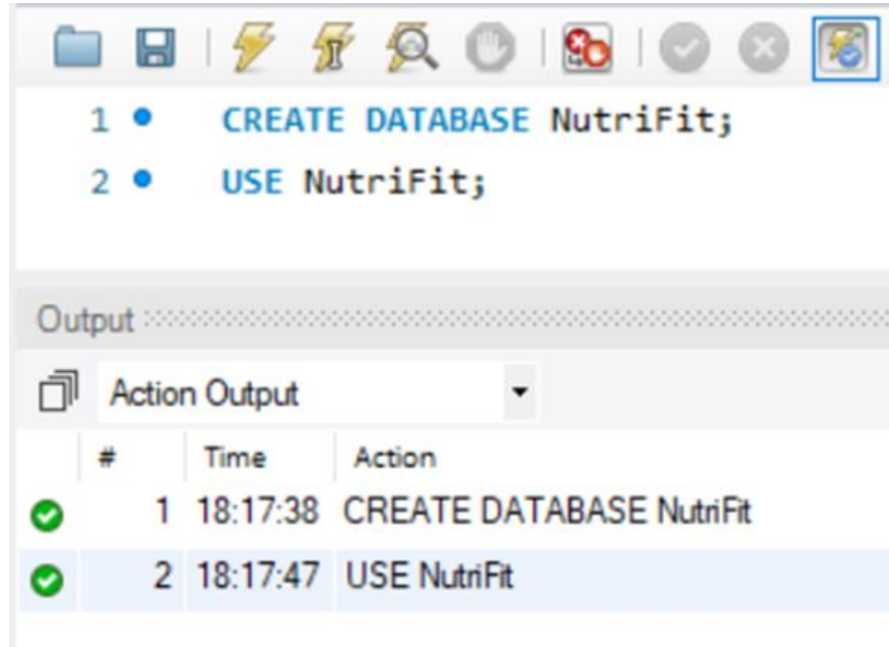


Fig. 1. Database Creation

1. **CREATE DATABASE** statement declares and initializes a new database in SQL. It is followed by the name of the Database: NutriFit.
2. The database can consists of tables, view, procedures etc. but to define them we must first select the specific database in which operations shall be performed. We use the **USE** statement to select a database.
3. The Action output tab records the success or failure of execution of a query. As shown below, the two statements are successfully executed.

Creation of Tables - Users, SupplierDetails, FoodInventory

1. DDL Statement - **CREATE TABLE** is used to define a new table (e.g. Users, SupplierDetails).
2. During the creation of the table the attributes are declared along with their **data types** (such as INT, VARCHAR(size), DECIMAL(total digits, decimal points)) and **constraints** (such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL)
3. **DESCRIBE** command displays the structure of table including, attributes, their data types and constraints.

```
3  -- Tables: FoodInventory, SupplierDetails, Users
4  • CREATE TABLE Users (
5      Userid INT PRIMARY KEY,
6      UserName VARCHAR(50),
7      UserEmail VARCHAR(100),
8      PreferredNutrient VARCHAR(20),
9      FoodPreference VARCHAR(10)
10 );
11 • DESCRIBE Users;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Userid	int	NO	PRI	NULL	
UserName	varchar(50)	YES		NULL	
UserEmail	varchar(100)	YES		NULL	
PreferredNutrient	varchar(20)	YES		NULL	
FoodPreference	varchar(10)	YES		NULL	

Fig. 2. Creation of Users Table

```
12 • CREATE TABLE SupplierDetails (
13     SupplierId INT PRIMARY KEY,
14     SupplierName VARCHAR(50),
15     SupplierType VARCHAR(30),
16     FoodCategory VARCHAR(30),
17     ShoppingPrice DECIMAL(10,2), -- Average price
18     Address VARCHAR(100)
19 );
20 • DESCRIBE SupplierDetails;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
SupplierId	int	NO	PRI	NULL	
SupplierName	varchar(50)	YES		NULL	
SupplierType	varchar(30)	YES		NULL	
FoodCategory	varchar(30)	YES		NULL	
ShoppingPrice	decimal(10,2)	YES		NULL	
Address	varchar(100)	YES		NULL	

Fig. 3. Creation of SupplierDetails Table

1. The FoodInventory table consists of FoodId as the Primary key which uniquely represents the table.
2. The table stores the Food items, their type (veg, non-veg etc.), their category (dairy, vegetables etc.) and they are declared as variable character data type.
3. The table also contains Purchase dates and Expiry dates with DATE datatype (yyyy-mm-dd format) that helps the application send expiry reminders to user.
4. Furthermore, the primary nutrient of the food, quantity of food items (in mL, L, g, kg) are also present.
5. FoodInventory table contains reference to the Users and SupplierDetails table with the help of **FOREIGN KEY** using their respective primary key as references.

```

21 CREATE TABLE FoodInventory (
22     FoodId INT PRIMARY KEY,
23     FoodName VARCHAR(20) UNIQUE NOT NULL,
24     FoodType VARCHAR(20), -- Veg, Non-veg, Vegan
25     FoodCategory VARCHAR(10),
26     PurchaseDate DATE,
27     ExpiryDate DATE,
28     PrimaryNutrient VARCHAR(20),
29     Quantity VARCHAR(10), -- ml, l, g, kg
30     SupplierId INT,
31     UserId INT,
32     FOREIGN KEY (SupplierId) REFERENCES SupplierDetails(SupplierId),
33     FOREIGN KEY (UserId) REFERENCES Users(UserId) );
34 DESCRIBE FoodInventory;

```

Field	Type	Null	Key	Default	Extra
FoodId	int	NO	PRI	NULL	
FoodName	varchar(20)	NO	UNI	NULL	
FoodType	varchar(20)	YES		NULL	
FoodCategory	varchar(10)	YES		NULL	
PurchaseDate	date	YES		NULL	
ExpiryDate	date	YES		NULL	
PrimaryNutrient	varchar(20)	YES		NULL	
Quantity	varchar(10)	YES		NULL	
SupplierId	int	YES	MUL	NULL	
UserId	int	YES	MUL	NULL	

Fig. 4. Creation of FoodInventory Table

Entity-Relationship Diagram

1. The ER diagram is a pictorial representation of the tables and relationships between them, it helps us understand how information is related and how it works together.
2. The cardinality between the tables is also defined, Users can store 1 or many food items hence (1 to N), but each food item is associated with one user hence (only one relationship), similarly Supplier can provide many food items (1 to N) but each food item is associated with one supplier (1 to 1).
3. The below diagram is created by using **Lucidchart** application tool.

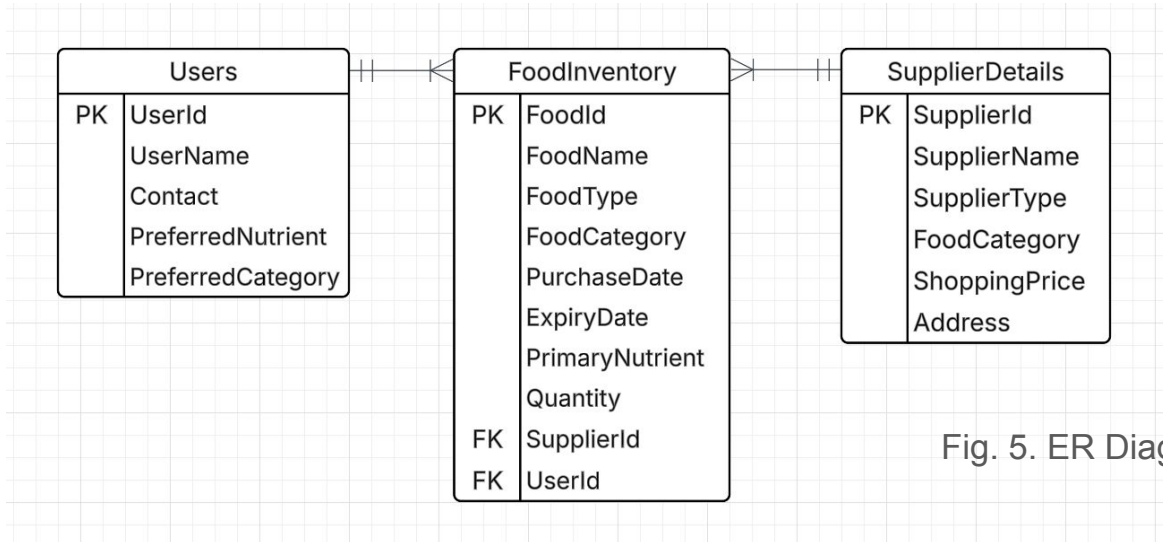


Fig. 5. ER Diagram of NutriFit

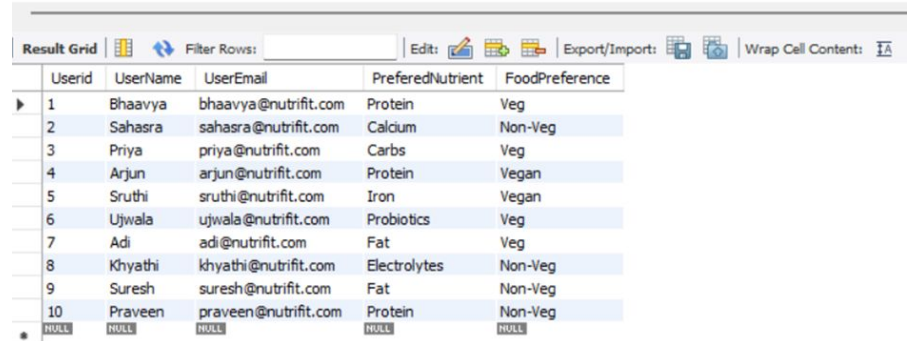
Insertion into Tables

1. **INSERT** is a DML statement used to insert data into a table. The data values are ordered according to the mentioned column names.
2. **SELECT** is a DQL statement to retrieve the required data from the table.
3. “*” indicates to retrieve all the data from the table. Whereas, using the **WHERE** clause we can retrieve selected statements only.
4. Here the 10 records of Users data - UserId, UserName, userEmail, PreferredNutrient, FoodPreference are inserted to the table.

SYNTAX:

```
INSERT INTO table_name (col1, col2 .... coln)  
VALUES (value1, value2,..... valuen);
```

```
35 -- Insertions  
36 • INSERT INTO Users (UserId, UserName, UserEmail, PreferredNutrient, FoodPreference)  
37 VALUES (1, 'Bhaavya', 'bhaavya@nutrfit.com', 'Protein', 'Veg'),  
38          (2, 'Sahasra', 'sahasra@nutrfit.com', 'Calcium', 'Non-Veg'),  
39          (3, 'Priya', 'priya@nutrfit.com', 'Carbs', 'Veg'),  
40          (4, 'Arjun', 'arjun@nutrfit.com', 'Protein', 'Vegan'),  
41          (5, 'Sruthi', 'sruthi@nutrfit.com', 'Iron', 'Vegan'),  
42          (6, 'Ujwala', 'ujwala@nutrfit.com', 'Probiotics', 'Veg'),  
43          (7, 'Adi', 'adi@nutrfit.com', 'Fat', 'Veg'),  
44          (8, 'Khyathi', 'khyathi@nutrfit.com', 'Electrolytes', 'Non-Veg'),  
45          (9, 'Suresh', 'suresh@nutrfit.com', 'Fat', 'Non-Veg'),  
46          (10, 'Praveen', 'praveen@nutrfit.com', 'Protein', 'Non-Veg');  
47 • SELECT * FROM Users;
```



	UserId	UserName	UserEmail	PreferredNutrient	FoodPreference
▶	1	Bhaavya	bhaavya@nutrfit.com	Protein	Veg
	2	Sahasra	sahasra@nutrfit.com	Calcium	Non-Veg
	3	Priya	priya@nutrfit.com	Carbs	Veg
	4	Arjun	arjun@nutrfit.com	Protein	Vegan
	5	Sruthi	sruthi@nutrfit.com	Iron	Vegan
	6	Ujwala	ujwala@nutrfit.com	Probiotics	Veg
	7	Adi	adi@nutrfit.com	Fat	Veg
	8	Khyathi	khyathi@nutrfit.com	Electrolytes	Non-Veg
	9	Suresh	suresh@nutrfit.com	Fat	Non-Veg
	10	Praveen	praveen@nutrfit.com	Protein	Non-Veg
•	NULL	NULL	NULL	NULL	NULL

Fig. 6. Insertion of data into Users Table

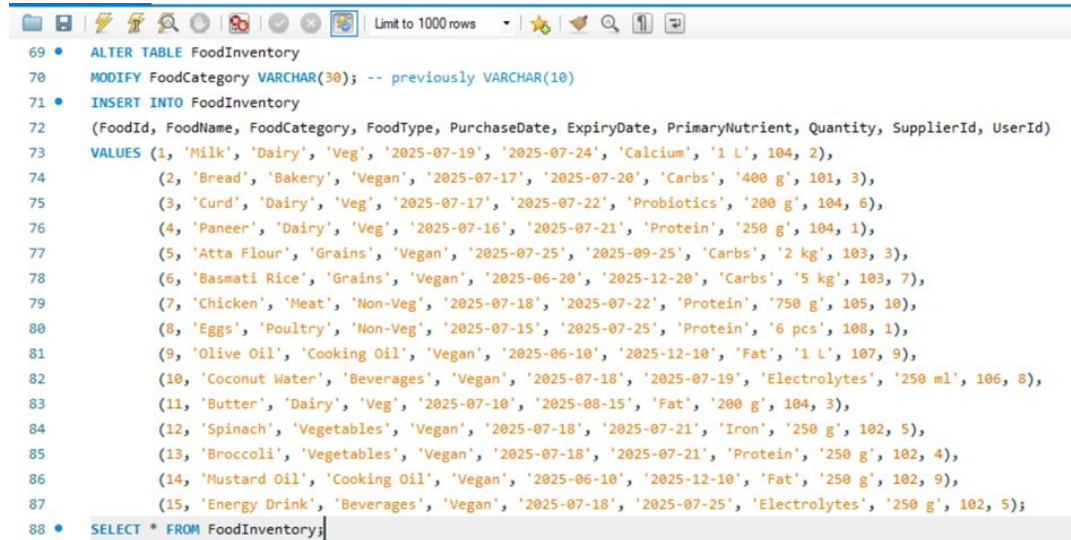
1. Similarly, 10 records of data is inserted using the **INSERT** statement into SupplierDetails table according to the respective column names - SupplierId, SupplierName, SupplierType (Organic, Wholesale, Retails), FoodCategory (Vegetables, Dairy etc.)
2. The **SELECT** statements help view the retrieved data in the results grid.

```
53 • INSERT INTO SupplierDetails (SupplierId, SupplierName, SupplierType, FoodCategory, ShoppingPrice, Address)
54 VALUES (101, 'Golden Bake', 'Retail', 'Bakery', 50.00, 'Kondapur, Hyderabad'),
55          (102, 'Puro Natural', 'Organic', 'Vegetables', 75.00, 'Banjara Hills, Hyderabad'),
56          (103, 'Metro', 'Wholesale', 'Grains', 500.00, 'Kukatpally, Hyderabad'),
57          (104, 'Polimeraas', 'Organic', 'Dairy', 60.00, 'Kondapur, Hyderabad'),
58          (105, 'Tender Cuts', 'Wholesale', 'Meat', 200.00, 'Gachibowli, Hyderabad'),
59          (106, 'Vijetha', 'Retail', 'Beverages', 150.00, 'Kondapur, Hyderabad'),
60          (107, 'Oil Mills', 'Organic', 'Cooking Oil', 300.00, 'Madhapur, Hyderabad'),
61          (108, 'Kinera', 'Organic', 'Poultry', 70.00, 'Miyapur, Hyderabad'),
62          (109, 'Ratandeep', 'Retail', 'Vegetables', 200.00, 'Kondapur, Hyderabad'),
63          (110, 'Golden groceries', 'Retail', 'Poultry', 100.00, 'Kondapur, Hyderabad');
64 • SELECT * FROM SupplierDetails;
65
```

SupplierId	SupplierName	SupplierType	FoodCategory	ShoppingPrice	Address
101	Golden Bake	Retail	Bakery	50.00	Kondapur, Hyderabad
102	Puro Natural	Organic	Vegetables	75.00	Banjara Hills, Hyderabad
103	Metro	Wholesale	Grains	500.00	Kukatpally, Hyderabad
104	Polimeraas	Organic	Dairy	60.00	Kondapur, Hyderabad
105	Tender Cuts	Wholesale	Meat	200.00	Gachibowli, Hyderabad
106	Vijetha	Retail	Beverages	150.00	Kondapur, Hyderabad
107	Oil Mills	Organic	Cooking Oil	300.00	Madhapur, Hyderabad
108	Kinera	Organic	Poultry	70.00	Miyapur, Hyderabad
109	Ratandeep	Retail	Vegetables	200.00	Kondapur, Hyderabad
110	Golden groceries	Retail	Poultry	100.00	Kondapur, Hyderabad
NULL	NULL	NULL	NULL	NULL	NULL

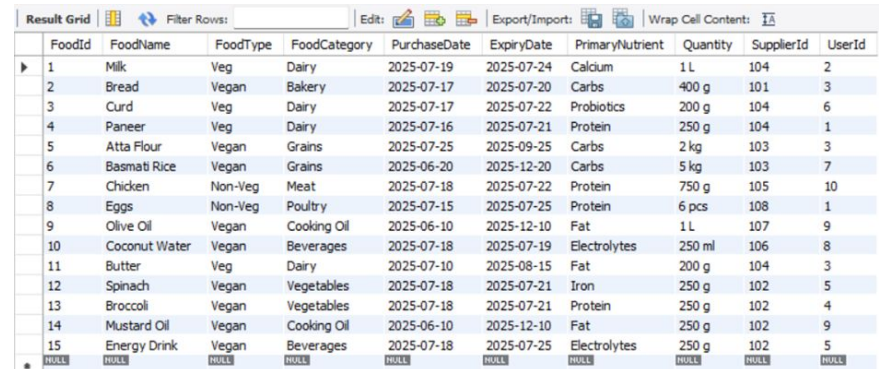
Fig. 7. Insertion of data into SupplierDetails Table

1. Similarly, 15 records data are inserted using the **INSERT** statement into FoodInventory tables according to the respective column names - FoodId, FoodCategory, PurchaseDate, ExpiryDate, PrimaryNutrient (Protein, Fat, Carbs etc.), Quantity and SupplierId and UserId are Foreign keys from SupplierDetails and Users table used to establish a connection between the tables.
2. During the creation of FoodInventory table, I miscalculated the size of FoodCategory data and assigned a small size, hence i used the **ALTER** statement to update the table my modifying my data type from VARCHAR(10) to VARCHAR(30) to fit larger data accordingly.
3. The **SELECT** statements help view the retrieved data in the results grid.



```

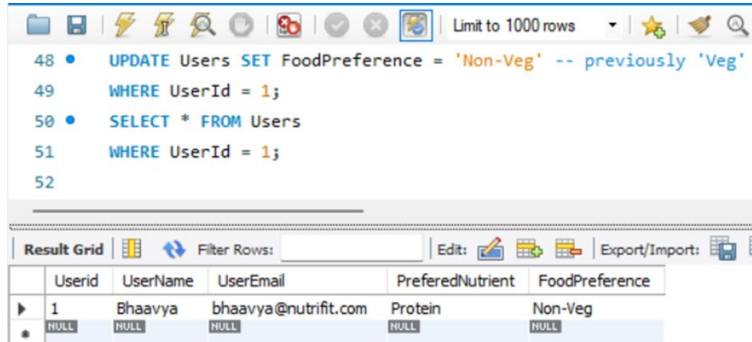
69 • ALTER TABLE FoodInventory
70 MODIFY FoodCategory VARCHAR(30); -- previously VARCHAR(10)
71 • INSERT INTO FoodInventory
72 (FoodId, FoodName, FoodCategory, FoodType, PurchaseDate, ExpiryDate, PrimaryNutrient, Quantity, SupplierId, UserId)
73 VALUES (1, 'Milk', 'Dairy', 'Veg', '2025-07-19', '2025-07-24', 'Calcium', '1 L', 104, 2),
74 (2, 'Bread', 'Bakery', 'Vegan', '2025-07-17', '2025-07-20', 'Carbs', '400 g', 101, 3),
75 (3, 'Curd', 'Dairy', 'Veg', '2025-07-17', '2025-07-22', 'Probiotics', '200 g', 104, 6),
76 (4, 'Paneer', 'Dairy', 'Veg', '2025-07-16', '2025-07-21', 'Protein', '250 g', 104, 1),
77 (5, 'Atta Flour', 'Grains', 'Vegan', '2025-07-25', '2025-09-25', 'Carbs', '2 kg', 103, 3),
78 (6, 'Basmati Rice', 'Grains', 'Vegan', '2025-06-20', '2025-12-20', 'Carbs', '5 kg', 103, 7),
79 (7, 'Chicken', 'Meat', 'Non-Veg', '2025-07-18', '2025-07-22', 'Protein', '750 g', 105, 10),
80 (8, 'Eggs', 'Poultry', 'Non-Veg', '2025-07-15', '2025-07-25', 'Protein', '6 pcs', 108, 1),
81 (9, 'Olive Oil', 'Cooking Oil', 'Vegan', '2025-06-10', '2025-12-10', 'Fat', '1 L', 107, 9),
82 (10, 'Coconut Water', 'Beverages', 'Vegan', '2025-07-18', '2025-07-19', 'Electrolytes', '250 ml', 106, 8),
83 (11, 'Butter', 'Dairy', 'Veg', '2025-07-10', '2025-08-15', 'Fat', '200 g', 104, 3),
84 (12, 'Spinach', 'Vegetables', 'Vegan', '2025-07-18', '2025-07-21', 'Iron', '250 g', 102, 5),
85 (13, 'Broccoli', 'Vegetables', 'Vegan', '2025-07-18', '2025-07-21', 'Protein', '250 g', 102, 4),
86 (14, 'Mustard Oil', 'Cooking Oil', 'Vegan', '2025-06-10', '2025-12-10', 'Fat', '250 g', 102, 9),
87 (15, 'Energy Drink', 'Beverages', 'Vegan', '2025-07-18', '2025-07-25', 'Electrolytes', '250 g', 102, 5);
88 • SELECT * FROM FoodInventory;
  
```



	FoodId	FoodName	FoodType	FoodCategory	PurchaseDate	ExpiryDate	PrimaryNutrient	Quantity	SupplierId	UserId
▶	1	Milk	Veg	Dairy	2025-07-19	2025-07-24	Calcium	1 L	104	2
	2	Bread	Vegan	Bakery	2025-07-17	2025-07-20	Carbs	400 g	101	3
	3	Curd	Veg	Dairy	2025-07-17	2025-07-22	Probiotics	200 g	104	6
	4	Paneer	Veg	Dairy	2025-07-16	2025-07-21	Protein	250 g	104	1
	5	Atta Flour	Vegan	Grains	2025-07-25	2025-09-25	Carbs	2 kg	103	3
	6	Basmati Rice	Vegan	Grains	2025-06-20	2025-12-20	Carbs	5 kg	103	7
	7	Chicken	Non-Veg	Meat	2025-07-18	2025-07-22	Protein	750 g	105	10
	8	Eggs	Non-Veg	Poultry	2025-07-15	2025-07-25	Protein	6 pcs	108	1
	9	Olive Oil	Vegan	Cooking Oil	2025-06-10	2025-12-10	Fat	1 L	107	9
	10	Coconut Water	Vegan	Beverages	2025-07-18	2025-07-19	Electrolytes	250 ml	106	8
	11	Butter	Veg	Dairy	2025-07-10	2025-08-15	Fat	200 g	104	3
	12	Spinach	Vegan	Vegetables	2025-07-18	2025-07-21	Iron	250 g	102	5
	13	Broccoli	Vegan	Vegetables	2025-07-18	2025-07-21	Protein	250 g	102	4
	14	Mustard Oil	Vegan	Cooking Oil	2025-06-10	2025-12-10	Fat	250 g	102	9
	15	Energy Drink	Vegan	Beverages	2025-07-18	2025-07-25	Electrolytes	250 g	102	5
•	total	total	total	total	total	total	total	total	total	total

Fig. 8. Insertion of data into FoodInventory Table

Modifying records in the tables



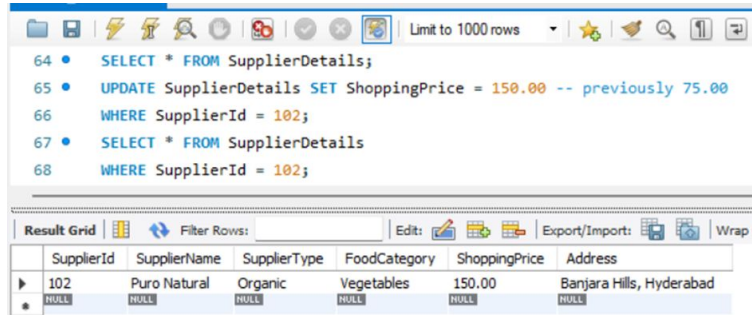
The screenshot shows a database client interface with a toolbar at the top. Below the toolbar, there is a SQL editor with the following commands:

```
48 • UPDATE Users SET FoodPreference = 'Non-Veg' -- previously 'Veg'
49 WHERE UserId = 1;
50 • SELECT * FROM Users
51 WHERE UserId = 1;
52
```

Below the SQL editor is a 'Result Grid' section. It includes a 'Filter Rows:' field and buttons for 'Edit', 'Export/Import', and 'Wrap'. The result grid displays the following data:

	UserId	UserName	UserEmail	PreferedNutrient	FoodPreference
▶	1	Bhaavya	bhaavya@nutrfit.com	Protein	Non-Veg
•	NULL	NULL	NULL	NULL	NULL

Fig. 9. Modifying record in Users Table



The screenshot shows a database client interface with a toolbar at the top. Below the toolbar, there is a SQL editor with the following commands:

```
64 • SELECT * FROM SupplierDetails;
65 • UPDATE SupplierDetails SET ShoppingPrice = 150.00 -- previously 75.00
66 WHERE SupplierId = 102;
67 • SELECT * FROM SupplierDetails
68 WHERE SupplierId = 102;
```

Below the SQL editor is a 'Result Grid' section. It includes a 'Filter Rows:' field and buttons for 'Edit', 'Export/Import', and 'Wrap'. The result grid displays the following data:

	SupplierId	SupplierName	SupplierType	FoodCategory	ShoppingPrice	Address
▶	102	Puro Natural	Organic	Vegetables	150.00	Banjara Hills, Hyderabad
•	NULL	NULL	NULL	NULL	NULL	NULL

Fig. 10. Modifying record in SupplierDetails Table

1. Databases allow us to update the data in table using **UPDATE**, a DML statement.
2. The user with UserId as 1, has changed their FoodPreference from 'Veg' to 'Non-Veg' using the **SET** command. While updating we must mention a specific record that requires the data change using the WHERE clause. Condition: **UserId = 1**
3. Using the SELECT statement we can view the update.
4. Similarly, in the SupplierDetails table, the ShoppingPrice attribute has been set to 150.00 from 75.00 using the UPDATE statement. The specific record is identified by WHERE clause condition: **SupplierId = 102**.

1. Similarly, the FoodInventory table is updated by changing the ExpiryDate attribute of record with **FoodId = 3** mentioned in WHERE clause.
2. The internal operation of modifying a record is, the database performs a **search operation** based on the WHERE condition and it identifies the matching records and updates the specified attributes with new values.
3. Without the WHERE clause all rows might be updated.

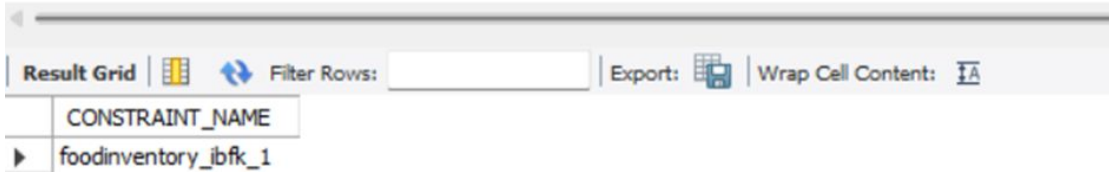
```
89 • UPDATE FoodInventory SET ExpiryDate = '2025-07-20' -- previously '2025-07-22'
90 WHERE FoodId = 3;
91 • SELECT * FROM FoodInventory
92 WHERE FoodId = 3;
93
```

Result Grid										
Filter Rows: Edit: Export/Import: Wrap Cell Content:										
	FoodId	FoodName	FoodType	FoodCategory	PurchaseDate	ExpiryDate	PrimaryNutrient	Quantity	SupplierId	UserId
▶	3	Curd	Veg	Dairy	2025-07-17	2025-07-20	Probiotics	200 g	104	6
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig. 11. Modifying record in FoodInventory Table

Dropping a table from the database

```
94  -- Dropping SupplierDetails table
95  -- Dropping the foreign key constraint in FoodInventory table
96  -- By finding the Constraint name in Information Schema
97  •  SELECT CONSTRAINT_NAME
98      FROM information_schema.KEY_COLUMN_USAGE
99      WHERE TABLE_NAME = 'FoodInventory' AND COLUMN_NAME = 'SupplierId';
100
```



CONSTRAINT_NAME
foodinventory_ibfk_1

Fig. 12. Getting Constraint name of foreign key

1. Let us consider a scenario where, the company NutriFit is relocating from Hyderabad to Chennai. Now it requires a new Supplier's details hence we are dropping that table.
2. But since the FoodInventory table and SupplierDetails table are connected using the SupplierId as FOREIGN KEY dropping the table will cause data loss in the other table.
3. Hence we must first **drop the FOREIGN KEY reference** from FoodInventory table.
4. The FOREIGN KEY is stored by a constraint name in the information schema so by identifying that we can drop the column.

Key Learning:

```
101 • ALTER TABLE FoodInventory
102 DROP FOREIGN KEY foodinventory_ibfk_1;
103
104 • SELECT * FROM FoodInventory
105 WHERE FoodId = 1;
```

Result Grid										
Filter Rows:										
Edit: Export/Import: Wrap Cell Content: I										
	FoodId	FoodName	FoodType	FoodCategory	PurchaseDate	ExpiryDate	PrimaryNutrient	Quantity	SupplierId	UserId
▶	1	Milk	Veg	Dairy	2025-07-19	2025-07-24	Calcium	1 L	104	2
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig. 13. Observation

1. One detail I observed while dropping the SupplierId FOREIGN KEY from FoodInventory table is, when I view the FoodInventory table the SupplierId attribute still remains in the table.
2. Dropping a Foreign key doesn't remove that attribute from the table, but just removes the foreign key constraint. This means, SupplierId attribute in FoodInventory is no longer linked to SupplierDetails table.
3. Hence, this operation doesn't remove the column SupplierId itself from FoodInventory table.

DROP TABLE SupplierDetails;

1. The **DROP** command is a DDL command which modifies the table (here, deletion)
2. Since, the foreign key link is removed the SupplierDetails can be removed safely.
3. The changes will be appeared in the schema tab after refreshing. As shown in the attached screenshot, the database NutriFit consists of only two tables: FoodInventory and Users.
4. This verifies that SupplierDetails is dropped from database.

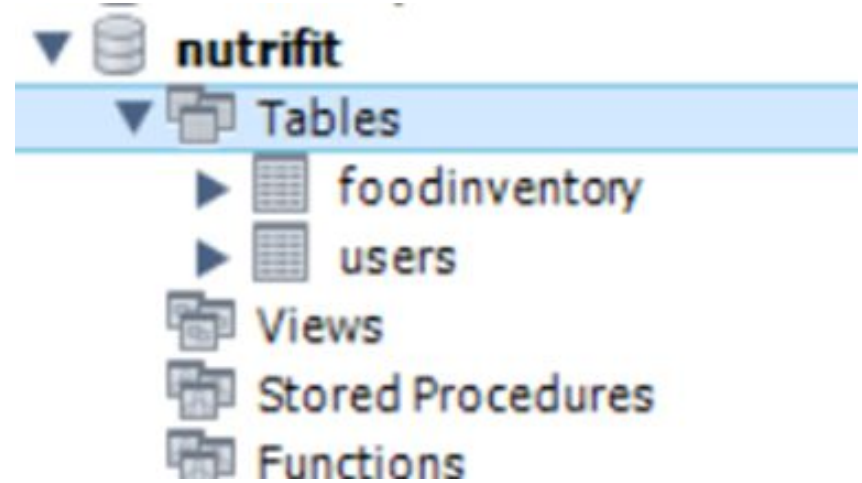


Fig. 14. Schema tab after dropping table

Updated E-R Diagram

1. After the deletion of SupplierDetails using DROP statement, the model's Entity Relationship diagram is updated consisting of only two entities: Users and FoodInventory. Observation has been made that SupplierID in FoodInventory table is no longer a FOREIGN KEY.
2. This screenshot is captured from the MySQL Workbench tool.

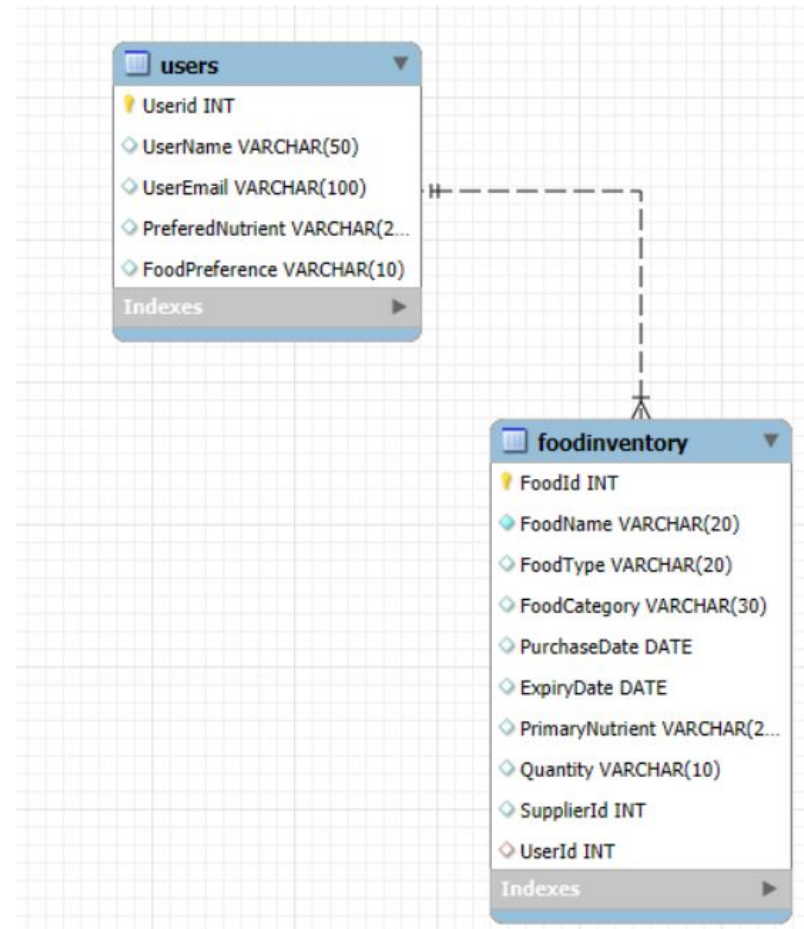


Fig. 15. Updated ER Diagram of NutriFit

References

1. Data Collection: [Food Expiration Date Guidelines Plus an Easy-to-Read Chart](#)
2. ER Diagram and LucidChart tutorial: [Entity Relationship Diagram \(ERD\) Tutorial - Part 1 - YouTube](#)

Thank You